Boolean Algebra, Identities, Proofs, and Minimization

Algorithms, Programming, and Data Representation · Module 92313 · Chapter 1a

From Fundamentals to AI Apocalypse · personal study guide

Contents

Start from memory 2

How to use this guide 2

How the ideas fit together 3

1.1 Foundations: one structure, several views 4

1.2 Core identities and proof tools 6

1.3 Completeness, normal forms, and minimization 7

1.4 Karnaugh maps remove changing variables 8

1.5 Shannon views, OBDDs, and algebraic normal form 9

1.6 Verification, hazards, and wrap-up 10

Formula map 11

Derive it cold 11

Explain it 11

Source checks 12

Mixed practice 13

Exam-style problems 14

Self-check 15

Answers 16

This guide develops Boolean logic through algebraic structures, proof tools, function minimization, and decision diagrams.

Start from memory

Answer before reading. Mark each response as sure, unsure, or a guess; uncertainty shows what to revisit.

1. Name the three operations and two constants in a Boolean algebra.
2. What is the dual of $x + 0 = x$?
3. State one absorption law.
4. What two cases does Shannon expansion split on?
5. How many literals remain in a K-map group of 8 cells for a 5-variable function?

Answers: see "Start from memory" under Answers at the end.

How to use this guide

Study one short section, close it, then reconstruct its laws or argument from memory. Mix proof, algebra, and diagram questions so you practise choosing the right tool.

1. **Day 0:** Try the cold start, then study §§1.1–1.2 and derive duality and De Morgan's laws.
2. **Day 1:** Rebuild the NAND proof and normal forms in §1.3; solve mixed practice 1–8.
3. **Day 3:** Draw K-map groupings from memory and solve practice 9–14.
4. **About day 7:** Reconstruct Shannon expansion, cofactors, and OBDD reduction; solve practice 15–20.
5. **About day 14:** Complete both exam-style problems without notes and explain the source checks.
6. **Before the exam:** Tick only skills you can show cold; return to any section referenced by a missed answer.

When stuck: name the definition, write one transformation at a time, and check the result on a small truth table. A guess counts as uncertain until you verify it.

How the ideas fit together

The same Boolean functions can be viewed as algebra, sets, rings, normal forms, maps, or decision diagrams. Each view makes a different task easier.

Define · identify the structure

- **Boolean algebra** §1.1

  join · meet · complement

  Axioms support equivalent algebraic and lattice views.

- **Identities and proofs** §1.2

  duality · De Morgan · absorption

  Derived laws shorten proofs and expressions.

Represent · choose a form

- **Complete connectives** §1.3

  NAND or NOR alone

  A complete set can express every Boolean function.

- **Karnaugh maps** §1.4

  adjacent cells → fewer literals

  Gray-code adjacency exposes variables that can be eliminated.

Analyse · split and verify

- **Shannon expansion** §1.5

  $f = x\ f_1 + x'\ f_0$

  Cofactors lead to decision diagrams and dependency tests.

- **Verification and hazards** §1.6

  same function · safe transitions

  Logical equivalence and physical timing answer different questions.

Question	Useful view
Can two formulas be shown equal?	Use axioms, derived identities, or a truth table.
Can a gate set express every function?	Show it can express NOT, AND, and OR.
Can a function be represented more compactly?	Try a normal form, a K-map, or a reduced OBDD.

1.1 Foundations: one structure, several views

A Boolean algebra packages truth operations into a system of equations. The same structure can also be understood through order, sets, or arithmetic modulo two.

Operations and axioms

Write the structure as (B, +, ·, ', 0, 1). Join (+) and meet (·) are binary operations; complement (') is unary. The Huntington-style axioms used in the lecture state commutativity, associativity, distributivity, identities, and complements.

Core axioms

$$x + y = y + x$$
$$(x + y) + z = x + (y + z)$$
$$x \cdot (y + z) = x \cdot y + x \cdot z$$
$$x + 0 = x$$
$$x \cdot 1 = x$$
$$x + x' = 1$$
$$x \cdot x' = 0$$

The lecture's axiom list also states the dual commutative and associative laws for meet. Other familiar laws can be derived.

Duality swaps + with · and 0 with 1. Every equation derivable from the axioms has a derivable dual; for example, $x + x = x$ has dual $x \cdot x = x$. A **distributive complemented bounded lattice** is the same structure in order language: join and meet have bottom 0 and top 1, and each element has a complement.

Worked proof · complements are unique

Suppose y behaves like a complement of x: x+y=1 and x·y=0. Show y=x'.

1. Use the identity and complement of x: y=y·1=y·(x+x').
2. Distribute: y=(y·x)+(y·x')=0+(y·x')=y·x'.
3. Similarly x'=x'·1=x'·(x+y)=x'·y. Therefore y=y·x'=x'·y=x'. §1.1

The proof relies on the stated complement conditions; it does not assume uniqueness in advance.

Sets, rings, and maps

The power set P(U), with union as +, intersection as ·, set complement as ', empty set as 0, and U as 1, is a Boolean algebra. There is also a ring view: XOR is addition, AND is multiplication, and every element is idempotent under multiplication.

Boolean ring translation

$$x \,\mathrm{XOR}\, y = (x \cdot y') + (x' \cdot y)$$
$$x \odot y = x \cdot y$$
$$x \wedge y = xy$$
$$x \vee y = x + y + xy$$
$$\neg x = 1 + x$$
$$x^2 = x$$
$$x + x = 0$$

The power-set example uses symmetric difference for ring addition and intersection for multiplication.

A homomorphism preserves the operations and constants. Its image is a subalgebra; when it is onto, the lecture's isomorphism theorem identifies the quotient by its kernel with the target algebra.

Trap: duality is a rule for transforming a proved equation; it does not mean complement every variable or reverse every term.

Recall check · answer without looking

1. Which operations and constants appear in the Boolean algebra structure?
2. How does the ring view interpret XOR and AND?

Answers: see "1.1 Foundations: one structure, several views" under Answers at the end.

1.2 Core identities and proof tools

Axioms are the starting point; identities are reusable consequences. Proofs become shorter when you name the law that justifies each step.

Law	Identity
De Morgan	$(x+y)'=x'\cdot y'(x\cdot y)'=x'+y'$
Absorption	$x+x\cdot y=xx\cdot(x+y)=x$
Consensus	$x\cdot y+x'\cdot z+y\cdot z=x\cdot y+x'\cdot z$
Shannon expansion	$f=x\cdot f_{x=1}+x'\cdot f_{x=0}$

De Morgan's law follows because $x'\cdot y'$ is the complement of $x+y$: their join is 1 and their meet is 0. The uniqueness-of-complements result from §1.1 then identifies it as $(x+y)'$. The second law follows by duality.

Worked simplification · remove a redundant condition

Simplify $x+x\cdot y$ using only laws from the lecture.

1. Factor x with distributivity: $x\cdot 1+x\cdot y=x\cdot(1+y)$.
2. Since $1+y=1$, this becomes $x\cdot 1$.
3. Use the identity law: $x\cdot 1=x$. Therefore $x+x\cdot y=x$. §1.2

Read the result as: once x is already included, the stricter case x and y adds no new true assignments.

Splitting a function into cases

The cofactor $f_{x=1}$ fixes x to 1; $f_{x=0}$ fixes it to 0. Shannon expansion inserts $x+x'=1$ and distributes f across the two cases. The Boolean difference is the XOR of the cofactors, so it is 1 exactly where changing x changes the output.

Difference and quantification

$$D_x f = f_{x=1} \operatorname{XOR} f_{x=0}$$
$$D_x(fg) = f\cdot D_{xg} \operatorname{XOR} g\cdot D_{xf} \operatorname{XOR} D_{xf}\cdot D_{xg}$$
$$(\exists x)f = f_{x=1} + f_{x=0}$$
$$(\forall x)f = f_{x=1}\cdot f_{x=0}$$

Existential quantification asks whether either case works; universal quantification asks whether both cases work. Both remove dependence on x.

Memory hook: for existential, OR the two cofactors; for universal, AND them. Check the question's quantifier before simplifying.

Recall check · answer without looking

1. Why can the complement uniqueness theorem prove De Morgan's first law?
2. What does a Boolean difference of 0 tell you about x?

Answers: see "1.2 Core identities and proof tools" under Answers at the end.

1.3 Completeness, normal forms, and minimization

A representation tells you how to build a function. Completeness says whether a chosen set of connectives can represent every function; minimization asks how small a representation can be.

Build every function from a small gate set

A connective set is functionally complete if it can express every Boolean function. NAND alone is complete because it can express NOT, AND, and OR. NOR alone is complete by the dual argument.

NAND construction

$$x \uparrow y = (x \cdot y)'$$
$$\neg x = x \uparrow x$$
$$x \cdot y = (x \uparrow y) \uparrow (x \uparrow y)$$
$$x + y = (x \uparrow x) \uparrow (y \uparrow y)$$

Once NOT, AND, and OR are available, any formula built from those connectives can be expressed with NAND.

Post's lattice classifies families of Boolean functions closed under composition. The lecture also contrasts monotone functions, which preserve order, with affine functions, which are parity expressions over F_2; a function that is both monotone and affine is constant.

Write and minimize formulas

Every Boolean function has a disjunctive normal form (OR of minterms for true rows) and a conjunctive normal form (AND of maxterms that exclude false rows). These forms are systematic, though a minimum exact two-level DNF or CNF search is NP-complete, so practical methods use structure and heuristics.

Worked example · XOR as DNF and CNF

Let f(A,B)=1 on rows 01 and 10, and 0 on rows 00 and 11.

1. For the true rows, the minterms are A'B and AB', so the DNF is A'B+AB'.
2. The false rows 00 and 11 give maxterms (A+B) and (A'+B'), so the CNF is (A+B)(A'+B').
3. Both forms describe XOR: the output is 1 exactly when the inputs differ. §1.3

Quine–McCluskey combines minterms that differ in one literal, repeats until prime implicants remain, then selects a cover. Its steps preserve coverage: every combined implicant covers only true assignments, and every listed true minterm remains covered.

Trap: a complete gate set guarantees expressiveness, not a small or fast circuit.

Recall check · answer without looking

1. How does NAND express NOT, AND, and OR?
2. What does DNF list, and why is finding a minimum exact form difficult?

Answers: see "1.3 Completeness, normal forms, and minimization" under Answers at the end.

1.4 Karnaugh maps remove changing variables

A K-map lays truth-table rows out so neighboring cells differ in one variable. Grouping adjacent 1s makes that changing variable disappear from the corresponding implicant.

For n variables, a map has 2^n cells in Gray-code order. A legal group contains 2^k adjacent cells, with wraparound adjacency allowed. Each grouped variable that changes across the group is eliminated; the implicant keeps $n-k$ literals. Use don't-care rows as either 0 or 1 when that makes a useful group.

K-map group rule

$$\text{Map size} : 2^n \text{ cells}$$
$$\text{Group size} : 2^k \text{ cells}$$
$$\text{Remaining literals} : n - k$$

Cover every required 1 at least once. Do not include a required 0. A don't-care can be included or ignored.

Worked example · simplify using an explicit bit order

Use variable order A,B,C with A as the most significant bit. Suppose f=1 for minterms 2, 3, 6, and 7.

1. These rows are 010, 011, 110, and 111, so B=1 in every true row.
2. On the map, they form a group of four. A and C vary; B stays fixed.
3. The group leaves one literal: f=B. Verify that B=0 rows map to 0 and B=1 rows map to 1. §1.4

Trap: write the variable bit order beside minterm indices. A minterm number has no meaning without a variable order.

Recall check · answer without looking

1. How many literals remain after grouping 8 cells on a 5-variable map?
2. What is the safe way to use a don't-care cell?

Answers: see "1.4 Karnaugh maps remove changing variables" under Answers at the end.

1.5 Shannon views, OBDDs, and algebraic normal form

Shannon expansion recursively splits a function into simpler cases. Reusing the same split structure leads to decision diagrams; a different basis expresses the function as parity over F_2.

Shannon expansion

$$f = x \cdot f_{x=1} + x' \cdot f_{x=0}$$

The selected input x chooses between the two cofactors, like a 2-to-1 multiplexer.

Worked example · expand on A

For f(A,B)=A·B+A', compute the two cofactors and reconstruct f.

1. Set A=1: $f_{A=1}$=B+0=B. Set A=0: $f_{A=0}$=0+1=1.
2. Substitute: f=A·B+A'·1.
3. The expansion matches the starting expression and separates the A=1 and A=0 cases. §1.5

An ordered binary decision diagram (OBDD) applies Shannon expansion in a fixed variable order. Reduction merges isomorphic subgraphs and removes tests whose two branches are identical. For one fixed order, the reduced OBDD is unique; a different variable order can produce a different diagram size.

The algebraic normal form (ANF) writes f as XORs of monomials over F_2. Every Boolean function has a unique ANF; its coefficients can be computed by a Möbius transform. Uniqueness follows because the monomials form a basis for all truth functions on n variables. Unlike DNF, ANF uses XOR as its addition operation.

ANF shape

$$f \text{ is the XOR of } a_S \cdot \prod_{i \in S} x_i \text{ over } S \subseteq [n]$$

Each coefficient a_S is 0 or 1. The empty product is 1; the XOR collects the selected monomials.

Trap: an OBDD is only canonical after you fix the variable order.

Recall check · answer without looking

1. How does a Shannon expansion combine its two cofactors?
2. What conditions make an OBDD reduced and canonical?

Answers: see "1.5 Shannon views, OBDDs, and algebraic normal form" under Answers at the end.

1.6 Verification, hazards, and wrap-up

Equivalent formulas can behave differently while a real circuit switches. Verification must check both the Boolean function and, where needed, the physical transitions.

Static-1 hazards

Consensus relation

$$xy + x'z + yz = xy + x'z$$

The term yz is logically redundant but can preserve coverage during a circuit transition.

A static-1 hazard occurs when the output should stay 1 for a small input change but a particular two-level sum-of-products implementation briefly drops to 0 because its paths have different delays. If two product terms hand off across one changing literal, their consensus term supplies overlapping coverage.

Worked example · add the consensus term

For f=xy+x'z, the consensus term is yz.

1. The terms xy and x'z differ on x; both can be 1 at the boundary when y=z=1.
2. Add yz. The consensus theorem gives xy+x'z+yz=xy+x'z, so the Boolean function is unchanged.
3. In a two-level sum-of-products implementation, yz keeps the output covered during the handoff and removes the static-1 hazard described in the lecture. §1.6

Soundness and completeness

Equational reasoning is sound when every derived equation is valid in every Boolean algebra model. It is complete for tautological equivalence when every such equivalence can be derived using the calculus. The lecture's wrap-up ties this proof view to the alternate ring, set, map, and decision-diagram representations. Its further-reading slide points to Stone on representation, Post on completeness, and Bryant on decision diagrams.

Sign check: Boolean equivalence says outputs agree for stable inputs; it does not guarantee zero propagation delay in hardware.

Recall check · answer without looking

1. Why can a redundant consensus term help a physical circuit?
2. What is the difference between soundness and completeness?

Answers: see "1.6 Verification, hazards, and wrap-up" under Answers at the end.

Formula map

Idea	Relation	Use
Duality	Swap + with · and 0 with 1	Derive the matching law.
De Morgan	(x+y)'=x'·y'(x·y)'=x'+y'	Move a complement across an operation.
Absorption	x+x·y=x	Remove a term already covered by x.
Consensus	xy+x'z+yz=xy+x'z	Remove a logically redundant term; add it back to cover a hazard.
Shannon	$f=xf_1+x'f_0$	Split on a variable.
Boolean difference	$D_x f=f_1$ XOR f_0	Test whether x can change the result.
K-map group	2^k cells leave n−k literals	Eliminate variables that change.
NAND	x↑y=(xy)'	Build NOT, AND, and OR from one gate type.

Derive it cold

Cover the right column and rebuild each chain using only earlier definitions and laws.

1. **Duality:** x+x=x → x·x=x. §1.1
2. **De Morgan:** show x'y' complements x+y, then use uniqueness. §1.2
3. **Absorption:** x+xy=x(1+y)=x. §1.2
4. **Shannon:** f=f(x+x')=xf+x'f, then read each term as a cofactor. §1.2
5. **NAND completeness:** derive NOT, AND, and OR, then cite functional completeness of those connectives. §1.3
6. **K-map grouping:** pair adjacent cells and show the changing variable disappears. §1.4
7. **OBDD uniqueness:** explain why the fixed order plus reduction rules determine the same graph. §1.5
8. **Hazard cover:** expand yz with x+x' and show it is absorbed by xy+x'z. §1.6

Explain it

Answer each "why" or "when" in your own words before opening the model response.

1. Why does duality save proof work?
2. Why is uniqueness of complements useful for proving De Morgan's law?
3. When is an apparently shorter Boolean expression unsafe to use as a circuit simplification?
4. Why can NAND alone express every Boolean function?
5. When should a don't-care be included in a K-map group?
6. Why does an OBDD's variable order matter even when its reduced form is canonical?
7. Why does a K-map group eliminate the variables that change?

Answers: see "Explain it" under Answers at the end.

Source checks

These two items were checked against the rendered slides. The guide states its indexing convention where it affects the result.

- **Slide 35 · minterm order not stated:** the slide gives $\Sigma m(1,3,5,7)$ and concludes f=B, but it does not label the variable bit order. Under the common A,B,C order with A most significant, that set describes C; the slide's conclusion matches a different bit order with B least significant. This guide states the convention explicitly in its worked example.
- **Slide 36 · expression conflicts with listed rows:** with W,X,Y,Z in that bit order, the listed 1s {1,3,7,11,15} and don't-cares {5,9,13} are all odd indices, while every even index is 0; the list therefore simplifies to f=Z. The rendered slide's printed expression includes W'Z', which is 1 on some even rows. This guide follows the listed rows and flags the mismatch for checking against the lecturer's source.

Mixed practice

Try all 20 questions in order without notes. The topics are interleaved; write your confidence beside each answer before checking the hidden key.

1. State the dual of x+0=x. §1.1
2. Simplify a+a·b. Name the law. §1.2
3. Using NAND ↑, write NOT x. §1.3
4. How many cells are on a 5-variable K-map? §1.4
5. For f(x,y)=xy+x', find $f_{x=1}$ and $f_{x=0}$. §1.5
6. What consensus term can be added to xy+x'z to cover a static-1 handoff? §1.6
7. In a Boolean ring, calculate 1+1 and 1·1. §1.1
8. Rewrite (a+b)' using De Morgan's law. §1.2
9. For f(A,B)=1 on rows 01 and 10 only, give one DNF. §1.3
10. A group of 8 cells is used on a 5-variable K-map. How many literals remain? §1.4
11. Expand f(A,B)=AB+A' on A by listing the two cofactors. §1.5
12. Distinguish soundness from completeness in equational reasoning. §1.6
13. Give the dual of x+x'=1. §1.1
14. What is Dx/Dx? §1.2
15. A function is both monotone and affine. What does the lecture say it must be? §1.3
16. With WXYZ as the bit order, use the slide 36 listed rows: 1s at 1,3,7,11,15 and don't-cares at 5,9,13. What is the simplest function? §1.4
17. Write A OR B in algebraic normal form over F_2. §1.5
18. Does adding yz change the Boolean function xy+x'z? §1.6
19. Why can exact minimum two-level DNF search require a heuristic in practice? §1.3
20. What does a Boolean difference of 0 with respect to x imply? §1.2

Answers: see "Mixed practice" under Answers at the end.

Exam-style problems

Allow about 20 minutes per problem. Show each identity used and check your final expression against the truth conditions.

1 · Prove, represent, and implement

For f(A,B,C)=A'B+AB'C, use the Boolean algebra and representation ideas from §§1.1–1.3.

1. Use Shannon expansion on A to write the two cofactors and reconstruct f.
2. Write a NAND-only implementation for each product term and the outer OR, using NAND as the only gate.
3. State whether the given expression is a DNF and explain how you would verify equivalence to your NAND version.

Answers: see "Exam-style problems" under Answers at the end.

2 · Minimize, then check a circuit transition

Let f(A,B,C)=1 for minterms 2,3,6,7 and 0 elsewhere. Use A,B,C as the bit order with A most significant.

1. Group the true cells on a 3-variable K-map and derive the minimal expression.
2. Expand your result on A using Shannon's theorem.
3. Explain what a static-1 hazard is and when the consensus theorem helps with it.

Answers: see "Exam-style problems" under Answers at the end.

Self-check

Use this list in a later session, at least a day after learning the material. Tick a skill only when you can demonstrate it without notes.

- ☐ State the Boolean operations, constants, and core axioms. (1, 7)
- ☐ Apply duality to a proved equation. (1, 13)
- ☐ Prove complement uniqueness and use it for De Morgan. (Derive it cold)
- ☐ Simplify with absorption and consensus. (2, 18)
- ☐ Compute cofactors, Shannon expansion, and Boolean difference. (5, 11, 14, 20)
- ☐ Distinguish existential and universal quantification.
- ☐ Translate XOR and AND in the Boolean ring view. (7, 17)
- ☐ Build NOT, AND, and OR from NAND. (3)
- ☐ Create DNF and CNF from truth conditions. (9)
- ☐ Explain why minimum exact forms can be computationally hard. (19)
- ☐ Group K-map cells and count the remaining literals. (4, 10, 16)
- ☐ State the variable order used by minterm indices. (16)
- ☐ Explain OBDD reduction and fixed-order canonicity.
- ☐ Write a simple function in algebraic normal form. (17)
- ☐ Distinguish Boolean equivalence from hazard-free switching. (6, 18)
- ☐ Rebuild the derivations from their definitions.
- ☐ Complete both multi-part problems and justify each step.

Answers

Start from memory

1. Join (+), meet (·), complement (’), and constants 0 and 1. §1.1
2. $x \cdot 1 = x$, swapping + with · and 0 with 1. §1.1
3. $x + x \cdot y = x$, or its dual. §1.2
4. It fixes the selected variable first to 1 and then to 0, combining the two cofactors. §1.5
5. Two literals remain: a group of 2^3 eliminates 3 of 5 variables. §1.4

1.1 Foundations: one structure, several views

1. Binary join +, binary meet ·, unary complement ’, and constants 0 and 1. §1.1
2. XOR is ring addition and AND is ring multiplication; each element satisfies x^2=x. §1.1

1.2 Core identities and proof tools

1. Show x’·y’ has join 1 and meet 0 with x+y; it is therefore the unique complement of x+y. §1.2
2. The two cofactors agree everywhere, so flipping x never changes f; f does not depend on x. §1.2

1.3 Completeness, normal forms, and minimization

1. NOT x=x↑x; AND x·y=(x↑y)↑(x↑y); OR x+y=(x↑x)↑(y↑y). §1.3
2. DNF ORs minterms for true rows. Exact minimum two-level minimization is NP-complete, so structured methods and heuristics matter. §1.3

1.4 Karnaugh maps remove changing variables

1. Five minus three changing variables leaves two literals. §1.4
2. Choose 0 or 1 if useful, but never use a don’t-care to cover a required zero. §1.4

1.5 Shannon views, OBDDs, and algebraic normal form

1. Multiply the x=1 cofactor by x, the x=0 cofactor by x’, then OR the two cases. §1.5
2. Fix a variable order, merge isomorphic subgraphs, and remove redundant tests; uniqueness is for that fixed order. §1.5

1.6 Verification, hazards, and wrap-up

1. It preserves overlapping coverage while unequal path delays could otherwise create a brief 1-to-0 glitch. §1.6
2. Soundness means every proved equation is valid. Completeness means every tautological equivalence is derivable. §1.6

Explain it

1. The duality principle preserves derivability when $+$ and $\cdot$, and 0 and 1, are exchanged. Once one equation is proved, its dual follows from the same axiom system without repeating the argument.
2. It is enough to verify that the proposed expression has join 1 and meet 0 with the original expression. Those two properties make it a complement, and uniqueness identifies it with the negation.
3. Logical equivalence alone does not account for unequal propagation delays. A two-level sum-of-products implementation may glitch during a transition, so a consensus term can be needed even when it is algebraically redundant.
4. NAND can produce NOT by tying its inputs together. It can then produce AND and OR, and those connectives can express every Boolean function.
5. Include it when it helps make a larger legal group and removes literals. Ignore it when it does not help; it is not a required 1 and must never force a group through a required 0.
6. Canonicity holds for a fixed order. Changing the order changes the sequence of Shannon splits and can therefore change the reduced diagram's shape and size.
7. The group contains every combination of the changing variables while the remaining variables stay fixed. Combining those cases with Boolean identities removes the changing literals, leaving only the fixed ones.

Mixed practice

1. x·1=x, swapping join with meet and 0 with 1. §1.1
2. a+a·b=a by absorption. §1.2
3. x'=x↑x because NAND of x with itself negates x. §1.3
4. 2^5=32 cells, one per input assignment. §1.4
5. $f_{x=1}$=y+0=y; $f_{x=0}$=0+1=1. §1.5
6. yz, giving xy+x'z+yz. The added term preserves the function and overlaps the handoff. §1.6
7. 1+1=0 by characteristic 2; 1·1=1 by the multiplicative identity. §1.1
8. (a+b)'=a'b'. §1.2
9. A'B+AB', the two minterms for rows 01 and 10. §1.3
10. Two literals: n−k=5−3=2. §1.4
11. $f_{A=1}$=B and $f_{A=0}$=1, so f=AB+A'·1. §1.5
12. Soundness means every derivable equation is valid; completeness means every tautological equivalence is derivable. §1.6
13. x·x'=0, the dual of x+x'=1. §1.1
14. 1: the cofactors of x with respect to itself are 1 and 0, whose XOR is 1. §1.2
15. It must be constant, as stated in the lecture. §1.3
16. Every odd index is either 1 or a don't-care and every even index is 0, so f=Z under the stated order. §1.4
17. A OR B = A XOR B XOR AB. Check rows 00, 01, 10, and 11 to verify the outputs 0,1,1,1. §1.5
18. No. The consensus theorem says xy+x'z+yz=xy+x'z. §1.6
19. Exact two-level minimization is NP-complete, so the search can grow too costly; structured methods and heuristics help. §1.3
20. The cofactors agree everywhere, so changing x never changes the function. §1.2

Exam-style problems

1. For A=1, f=B'C; for A=0, f=B. Therefore f=A·(B'C)+A'·B, the original function after reordering its terms. §1.2 §1.5
2. Make complements with x'=x↑x. For p=A'B, compute u=A'↑B=(A'B)', then p=u↑u. For q=AB'C, compute v=A↑B'=(AB')', r=v↑v=AB', s=r↑C=(AB'C)', then q=s↑s. Finally f=(p↑p)↑(q↑q), which is p OR q by De Morgan. §1.3
3. Yes: it is an OR of product terms. A truth table or algebraic rewriting can show that the gate expression produces the same value on every assignment. §1.3

Exam-style problems

1. The rows 010, 011, 110, and 111 all have B=1; they form a group of four. A and C vary, so f=B. §1.4
2. B is independent of A, so both A cofactors equal B. Shannon gives f=AB+A'B=B(A+A')=B. §1.2 §1.5
3. A static-1 hazard is a brief drop to 0 when the intended output stays 1 during a small input change. In a two-level sum-of-products implementation, an added consensus term can maintain overlapping coverage without changing the Boolean function. §1.6

Study guide based on Bernhard Kainz, "Boolean Algebra, Identities, Proofs, and Minimization: An advanced mathematical perspective for CS undergraduates," supplied as APDR lecture01 - extended.pdf (48 slides, dated 5 October 2025). Written in my own words; no lecture pages or figures are reproduced. Source inconsistencies checked against rendered slides 35–36 are described above. Not an official FAU publication; check course details against the current materials.